

Rapport détaillé de réalisation

Module NoSQL Security — Séance 3

Authentification, RBAC, exposition réseau et hygiène de configuration

Objet du document. Ce rapport reconstruit le travail effectivement réalisé à partir du support de séance en HTML et de l'archive de captures d'écran fournie. L'objectif est de produire un livrable exploitable tel quel, rédigé en LaTeX / LuaLaTeX, propre à l'impression et suffisamment détaillé pour documenter l'exécution des travaux pratiques, les tests autorisés / refusés et l'état réel du hardening observé.

Support pédagogique	<code>seance3_nosql_security_Partie3(2).html</code>
Preuves visuelles	<code>Lecture 3 Screenshots.zip</code> (16 captures)
Base de travail	<code>soc_lite</code>
Environnement visé	Docker + MongoDB 7 / mongosh
Livrables produits	PDF + code source LuaLaTeX

Table des matières

1	Objet du rapport	2
2	Sources et méthode de reconstruction	2
2.1	Corpus exploité	2
2.2	Approche d'analyse	2
3	Environnement et périmètre technique	3
4	Chronologie détaillée des travaux réalisés	3
4.1	Phase 1 – Démonstration d'une instance MongoDB insecure	3
4.2	Phase 2 – Activation de l'authentification et validation du compte administrateur .	4
4.3	Phase 3 – Validation de la création manuelle de l'administrateur initial	5
4.4	Phase 4 – Validation de l'authenticationDatabase	5
4.5	Phase 5 – Construction de la politique RBAC	6
4.5.1	Création du rôle personnalisé <code>incidentManager</code>	6
4.5.2	Insertion des données de test	6
4.5.3	Création des comptes applicatifs et métiers	6
4.6	Phase 6 – Tests d'accès par profil	7
4.6.1	<code>soc_analyst_ro</code> : lecture autorisée, écriture refusée, mais lecture de <code>users</code> possible	7
4.6.2	<code>soc_manager</code> : rôle personnalisé correctement borné	7
4.6.3	<code>soc_app</code> : fonctionnement applicatif validé, mais privilèges trop larges . . .	7
5	Analyse “avant / après” du durcissement	8
5.1	Avant durcissement	8
5.2	Après durcissement	8
6	Checklist de durcissement et état d'avancement	8
7	Évaluation finale	9
7.1	Ce qui est pleinement réussi	9
7.2	Ce qui reste à finaliser pour un moindre privilège strict	9
8	Conclusion	10
A	Annexe A – Journal de preuves visuelles	11

1 Objet du rapport

Le présent rapport documente le travail réalisé dans le cadre de la **Séance 3 – Hardening MongoDB**. L’objectif de la séance était de démontrer, sur une base NoSQL de type SOC, la différence entre une instance MongoDB exposée sans authentification et une instance correctement durcie par :

- l’activation de l’authentification,
- la création d’un administrateur initial,
- la gestion correcte de la base d’authentification (`authenticationDatabase`),
- la mise en place d’un contrôle d’accès par rôles (RBAC),
- la validation explicite des opérations *autorisées* et *refusées*.

Le support pédagogique attendait un mini-rapport “avant / après” ainsi qu’une checklist de hardening. Ce document remplit cette finalité et ajoute une analyse de cohérence entre la politique visée et les résultats réellement observés.

2 Sources et méthode de reconstruction

2.1 Corpus exploité

Deux sources ont été exploitées :

1. le support HTML de la séance, qui définit les objectifs, les activités, les commandes attendues et les contrôles à vérifier ;
2. les 16 captures d’écran fournies, qui permettent de reconstituer chronologiquement les manipulations effectivement exécutées.

2.2 Approche d’analyse

La reconstruction a été menée selon la logique suivante :

- ordonner les captures par horodatage ;
- identifier, pour chaque capture, la commande exécutée, le contexte MongoDB visé et le résultat obtenu ;
- regrouper les preuves en trois blocs : **exposition insecure**, **authentification**, **RBAC / moindre privilège** ;
- comparer les résultats observés avec la cible de sécurité du support ;
- signaler les écarts significatifs, notamment la limite des rôles built-in `read` et `readWrite` sur la collection `users`.

Synthèse immédiate. Le jeu de preuves fourni documente de manière convaincante l'exécution des trois activités principales du support :

1. démonstration d'un MongoDB *insecure* ;
2. activation et validation de l'authentification ;
3. conception et test d'une politique RBAC.

En revanche, la **correction finale du devoir maison** – à savoir la création de rôles personnalisés supplémentaires pour empêcher l'accès à **users** aux comptes **soc_app** et **soc_analyst_ro** – n'apparaît pas comme exécutée dans les captures. Cette partie reste donc **à compléter**.

3 Environnement et périmètre technique

Le travail est réalisé dans un environnement Docker avec **mongosh**. La base cible est **soc_lite**. Les objets manipulés au cours des démonstrations sont :

- **security_logs** : journaux de sécurité,
- **incidents** : incidents SOC,
- **users** : référentiel utilisateurs.

Trois profils fonctionnels sont ensuite modélisés :

- **soc_app** : compte applicatif de collecte,
- **soc_analyst_ro** : analyste en lecture,
- **soc_manager** : responsable incidents.

4 Chronologie détaillée des travaux réalisés

4.1 Phase 1 – Démonstration d'une instance MongoDB insecure

La première séquence de captures montre le lancement d'une instance MongoDB sans authentification et avec exposition réseau large. Cette phase répond à l'objectif pédagogique de visualiser le risque d'un service accessible sans contrôle.

Extrait de commandes observées

```
docker run -d --name mongo_insecure -p 27017:27017 mongo:7
docker exec -it mongo_insecure mongosh

use soc_lite
db.security_logs.find().limit(3)
db.incidents.drop()
db.dropDatabase()
```

Constats techniques

- Le port 27017 est publié sans restriction d'interface via **-p 27017:27017**.
- La connexion à **mongosh** est réalisée **sans identifiants**.
- Des opérations destructrices sont acceptées : suppression de la collection **incidents**, puis suppression de la base **soc_lite**.

Résultat de sécurité. Cette séquence établit clairement qu'une instance MongoDB démarrée sans authentification autorise la lecture, l'écriture et même la destruction de données par tout client ayant accès au port d'écoute.

Preuve complémentaire d'écriture et d'administration anonymes

Une seconde capture confirme que l'accès anonyme permet non seulement la lecture mais aussi l'insertion et l'administration du serveur :

Commandes observées

```
use soc_lite
db.security_logs.insertOne({
  timestamp: new Date(),
  source_ip: "10.0.0.99",
  event_type: "test_insecure",
  message: "Ce document a été inséré sans authentification"
})

db.security_logs.find({ event_type: "test_insecure" })
db.adminCommand({ listDatabases: 1 })
```

Le retour `acknowledged: true` ainsi que la réussite de `listDatabases` prouvent qu'un client non authentifié possède un contrôle complet sur l'instance.

4.2 Phase 2 – Activation de l'authentification et validation du compte administrateur

La séquence suivante montre le basculement vers une instance sécurisée.

Commandes observées (version sécurisée)

```
docker rm -f mongo_insecure

docker run -d --name mongo_secure \
  -p 127.0.0.1:27017:27017 \
  -e MONGO_INITDB_ROOT_USERNAME=<admin> \
  -e MONGO_INITDB_ROOT_PASSWORD=<mot_de_passe_admin> \
  mongo:7

docker exec -it mongo_secure mongosh
use soc_lite
db.security_logs.find()
```

Constats techniques

- Le port est cette fois lié à 127.0.0.1, ce qui réduit l'exposition réseau côté hôte.
- Toute opération sur les données retourne `MongoServerError[Unauthorized]: Command find requires authentication`.
- La protection n'est donc plus théorique : elle est validée par un refus explicite.

L'administrateur initial est ensuite utilisé avec succès :

Validation du compte administrateur

```
docker exec -it mongo_secure mongosh -u <admin> -p <mot_de_passe_admin>
db.adminCommand({ listDatabases: 1 })
```

Le résultat montre la liste des bases disponibles, ce qui confirme :

1. que l'authentification est activée;
2. que le compte administrateur initial fonctionne;
3. que la protection distingue désormais clairement un accès anonyme d'un accès authentifié.

4.3 Phase 3 – Validation de la création manuelle de l'administrateur initial

Une autre séquence montre une variante intéressante : la création manuelle de l'administrateur sur un conteneur lancé avec `mongod --auth --bind_ip_all`. Cette partie démontre la bonne compréhension du mécanisme de *localhost exception*.

Séquence manuelle observée

```
docker run -d --name mongo_manual -p 127.0.0.1:27017:27017 mongo:7 \
  mongod --auth --bind_ip_all

docker exec -it mongo_manual mongosh
use admin
db.createUser({
  user: "<admin>",
  pwd: "<mot_de_passe_admin>",
  roles: [{ role: "root", db: "admin" }]
})

quit()
docker exec -it mongo_manual mongosh -u <admin> -p <mot_de_passe_admin>
db.adminCommand({ listDatabases: 1 })
```

Intérêt de cette étape. Cette démonstration va au-delà de la simple utilisation des variables d'environnement Docker : elle montre que l'étudiant a également validé le scénario pédagogique de création manuelle du premier compte root.

4.4 Phase 4 – Validation de l'authenticationDatabase

La notion de `authenticationDatabase` est explicitement testée dans les captures. Un compte `soc_app` est utilisé avec la bonne base d'authentification, puis avec une base erronée.

Tests observés

```
docker exec -it mongo_manual mongosh -u soc_app -p <mot_de_passe_app> \
  --authenticationDatabase soc_lite

use soc_lite
db.security_logs.find()
```

La connexion réussit lorsque la base d'authentification est `soc_lite`.

Erreur observée avec mauvaise authDB

```
docker exec -it mongo_manual mongosh -u soc_app -p <mot_de_passe_app> \
--authenticationDatabase admin
```

MongoServerError: Authentication failed.

Conclusion de la phase. La preuve est nette : le couple (username, authenticationDatabase) fait partie intégrante de l'identité MongoDB. La compréhension de ce point est démontrée opérationnellement.

4.5 Phase 5 – Construction de la politique RBAC

4.5.1 Création du rôle personnalisé incidentManager

L'étape suivante consiste à créer un rôle personnalisé limité à la collection `incidents` :

Création du rôle personnalisé

```
use soc_lite
db.createRole({
  role: "incidentManager",
  privileges: [
    {
      resource: { db: "soc_lite", collection: "incidents" },
      actions: ["find", "insert", "update", "remove"]
    }
  ],
  roles: []
})
```

Cette étape est structurante pour la suite : elle matérialise le passage d'un modèle de droits générique à un modèle de droits finement ciblé.

4.5.2 Insertion des données de test

Les captures montrent ensuite l'insertion d'un jeu de données cohérent pour pouvoir tester les droits :

- trois événements dans `security_logs` (échecs et succès SSH),
- deux incidents dans `incidents`,
- deux utilisateurs dans `users`.

Les objets visibles incluent notamment :

- l'incident INC-2026-0042 : brute-force SSH sur `srv-web-01`,
- l'incident INC-2026-0041 : scan de ports depuis 192.168.5.200,
- les utilisateurs `analyst_dupont` et `analyst_martin`.

4.5.3 Création des comptes applicatifs et métiers

Les comptes suivants sont observés :

- `soc_analyst_ro` avec le rôle built-in `read` sur `soc_lite`,
- `soc_manager` avec le rôle personnalisé `incidentManager`,

— `soc_app` avec le rôle built-in `readWrite` sur `soc_lite`.

Les commandes sont visibles dans les captures et confirmées par `db.getUsers()` et `db.getRoles()`.

4.6 Phase 6 – Tests d'accès par profil

4.6.1 `soc_analyst_ro` : lecture autorisée, écriture refusée, mais lecture de users possible

Les preuves montrent la séquence suivante :

- lecture des logs : **autorisée**,
- lecture des incidents de sévérité `high` : **autorisée**,
- insertion dans `security_logs` : **refusée**,
- suppression d'un incident : **refusée**,
- lecture de `users` : **autorisée en pratique**.

Observation critique. Le support pédagogique visait une politique où l'analyste n'accède pas à `users`. Or la capture montre que `db.users.find()` retourne bien les deux documents. Ce comportement est cohérent avec MongoDB : le rôle built-in `read` sur une base s'applique à **toutes les collections** de cette base. Il n'est donc pas suffisamment restrictif pour satisfaire le moindre privilège sur `soc_lite`.

4.6.2 `soc_manager` : rôle personnalisé correctement borné

La séquence pour `soc_manager` est la plus conforme à la politique cible :

- lecture des incidents : **autorisée**,
- mise à jour de `INC-2026-0042` : **autorisée** (`modifiedCount: 1`),
- lecture des logs : **refusée**,
- lecture de `users` : **refusée**,
- commande d'administration `listDatabases` : **refusée**.

Évaluation. Le rôle personnalisé `incidentManager` réalise effectivement une séparation propre des privilèges : accès en lecture/écriture sur `incidents` uniquement, sans débordement vers les logs, les utilisateurs ou l'administration du serveur.

4.6.3 `soc_app` : fonctionnement applicatif validé, mais privilèges trop larges

La séquence observée pour `soc_app` montre :

- insertion d'un nouvel événement `firewall_drop` dans `security_logs` : **autorisée**,
- lecture des incidents ouverts : **autorisée**,
- lecture de `users` : **autorisée en pratique**,
- commande `listDatabases` : **refusée**.

Point de sécurité restant. Le compte `soc_app` ne peut pas administrer l'instance, ce qui est positif. En revanche, le rôle built-in `readWrite` sur `soc_lite` lui donne aussi accès à `users`. Le fonctionnement applicatif est donc validé, mais la politique de moindre privilège n'est pas totalement atteinte.

5 Analyse “avant / après” du durcissement

5.1 Avant durcissement

Élément	État observé
Authentification	Absente ; connexion anonyme possible
Port MongoDB	Publié sans restriction via <code>-p 27017:27017</code>
Lecture de données	Libre, sans identifiants
Écriture de données	Libre, sans identifiants
Suppression / destruction	Possible via <code>drop()</code> et <code>dropDatabase()</code>
Administration	Possible anonymement via <code>listDatabases</code>

5.2 Après durcissement

Élément	État observé
Authentification	Activée ; accès anonyme refusé
Port MongoDB	Restreint à <code>127.0.0.1</code> côté publication Docker
Compte root	Fonctionnel pour l'administration
authDB	Testée avec succès et échec contrôlé
RBAC personnalisé	Mis en place pour <code>soc_manager</code>
Séparation applicative	Partielle : <code>soc_manager</code> bien borné, <code>soc_analyst_ro</code> et <code>soc_app</code> trop larges via rôles built-in
Administration par comptes non-root	Refusée

6 Checklist de durcissement et état d'avancement

Contrôle	État	Justification fondée sur les preuves
Démonstration du risque MongoDB sans auth	Oui	Lecture, insertion, suppression de collection et <code>dropDatabase()</code> observés sans authentification
Activation de l'authentification	Oui	Refus explicite des commandes anonymes
Création d'un compte administrateur initial	Oui	Compte root validé sur instance sécurisée et instance manuelle
Validation de <code>authenticationDatabase</code>	Oui	Succès sur <code>soc_lite</code> et échec sur <code>admin</code> pour <code>soc_app</code>

Restriction de l'exposition réseau	Oui	Publication Docker liée à 127.0.0.1
Création d'un rôle personnalisé fin	Oui	incidentManager créé et testé
Création d'un profil analyste en lecture	Partiel	Lecture et refus d'écriture validés, mais accès à users encore possible
Création d'un profil manager limité aux incidents	Oui	Accès incidents validé, logs/users/admin refusés
Création d'un profil applicatif limité	Partiel	Fonction applicative validée, mais accès à users encore possible
Gestion des secrets via .env / externalisation	Non démontré	Aucune preuve fournie dans les captures
Checklist signée	Non démontré	Non visible dans les éléments transmis
Devoir maison de correction finale des rôles built-in	Non démontré	Pas de preuve d'une remédiation effective pour exclure users

7 Évaluation finale

7.1 Ce qui est pleinement réussi

Le travail remis montre une maîtrise solide des fondamentaux opérationnels de sécurité MongoDB :

- l'étudiant a **prouvé** le danger d'une instance sans authentification ;
- il a **basculé vers une instance protégée** et vérifié le refus d'accès anonyme ;
- il a **créé et utilisé** un administrateur initial ;
- il a **validé la notion d'authDB** avec un test positif et un test négatif ;
- il a **construit un rôle personnalisé pertinent** pour la gestion des incidents ;
- il a **testé systématiquement** les opérations autorisées et refusées selon chaque profil.

7.2 Ce qui reste à finaliser pour un moindre privilège strict

Le point principal restant concerne la collection **users**. Les rôles built-in **read** et **readWrite** sont trop larges pour satisfaire la politique métier visée. Pour achever le devoir de durcissement, il faudrait :

1. remplacer **read** pour **soc_analyst_ro** par un rôle personnalisé limitant la lecture à **security_logs** et **incidents** ;
2. remplacer **readWrite** pour **soc_app** par un rôle personnalisé limitant l'écriture à **security_logs** et, si nécessaire, la lecture ciblée de **incidents** ;
3. conserver **users** hors du périmètre des comptes applicatifs et métiers ;
4. externaliser les mots de passe dans un **.env** ou un gestionnaire de secrets au lieu de les laisser visibles en ligne de commande.

Formulation honnête de conclusion. Sur la base des preuves fournies, le **TP principal est réalisé avec succès**. La partie **devoir maison** est en revanche **seulement préparée** par l'observation du problème, mais **non démontrée comme effectivement terminée** dans les captures reçues.

8 Conclusion

Ce rapport met en évidence une progression nette entre un état initial *insecure* et un état partiellement durci, avec authentification, compte administrateur, contrôle d'accès RBAC et tests de validation. La réalisation est techniquement sérieuse et pédagogique, car elle ne se limite pas à “faire marcher” les commandes : elle vérifie les effets attendus et les refus de sécurité.

Le résultat le plus important, du point de vue cybersécurité, est aussi le plus instructif : les rôles built-in semblent pratiques, mais ils ne suffisent pas toujours à respecter le *least privilege*. Le travail fourni le démontre concrètement. C'est précisément ce type de constat qui transforme un TP en apprentissage de sécurité utile.

A Annexe A – Journal de preuves visuelles

Cette annexe reprend l'intégralité des captures fournies et les annote de manière synthétique. L'objectif est de conserver la traçabilité des manipulations réellement exécutées.

```
C:\Users\zguio>docker run -d --name mongo_insecure -p 27017:27017 mongo:7
a7779453a70e2c04e7f13bef0ef0ddaf46da6d7a7ea5b85a2d2ce419a5fd42be

C:\Users\zguio>docker exec -it mongo_insecure mongosh
Current Mongosh Log ID: 69da8e02a8c5d3e62d8563b0
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS
=2000&appName=mongosh+2.7.0
Using MongoDB:      7.0.30
Using Mongosh:      2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically
(https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
  The server generated these startup warnings when booting
    2026-04-11T18:07:41.135+00:00: Using the XFS filesystem is strongly recommended with the Wired
    Tiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
    2026-04-11T18:07:41.417+00:00: Access control is not enabled for the database. Read and write
    access to data and configuration is unrestricted
  -----

test> use soc_lite
switched to db soc_lite
soc_lite> db.security_logs.find().limit(3)

soc_lite> db.incidents.drop()
true
soc_lite> db.dropDatabase()
{ ok: 1, dropped: 'soc_lite' }
```

FIGURE 1 – Capture 1 – Instance mongo_insecure et destruction anonyme de la base.

Commentaire d'analyse. La capture montre le lancement d'un conteneur MongoDB sans authentification, la connexion anonyme via mongosh, puis des opérations destructrices acceptées : `db.incidents.drop()` retourne `true` et `db.dropDatabase()` supprime `soc_lite`. C'est la preuve la plus forte du risque initial.

```
C:\Users\zguio>docker run -d --name mongo_insecure -p 27017:27017 mongo:7
537ccf038e9c4a441e2691293115b484328eca5ca7c0bdd690cb7cbabd849691

C:\Users\zguio>docker exec -it mongo_insecure mongosh
Current Mongosh Log ID: 69da8f0bcf613544a18563b0
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS
=2000&appName=mongosh+2.7.0
Using MongoDB:      7.0.30
Using Mongosh:      2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically
(https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
 2026-04-11T18:12:19.625+00:00: Using the XFS filesystem is strongly recommended with the Wired
Tiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
 2026-04-11T18:12:19.820+00:00: Access control is not enabled for the database. Read and write
access to data and configuration is unrestricted
-----

test> use soc_lite
switched to db soc_lite
soc_lite> db.security_logs.insertOne({
|   timestamp: new Date(),
|   source_ip: "10.0.0.99",
|   event_type: "test_insecure",
|   message: "Ce document a été inséré sans authentification"
| })
{
  acknowledged: true,
  insertedId: ObjectId('69da8f1fcf613544a18563b1')
}
soc_lite> db.security_logs.find({ event_type: "test_insecure" })
[
  {
    _id: ObjectId('69da8f1fcf613544a18563b1'),
    timestamp: ISODate('2026-04-11T18:12:47.628Z'),
    source_ip: '10.0.0.99',
    event_type: 'test_insecure',
    message: 'Ce document a été inséré sans authentification'
  }
]
soc_lite> db.adminCommand({ listDatabases: 1 })
{
  databases: [
    { name: 'admin', sizeOnDisk: Long('8192'), empty: false },
    { name: 'config', sizeOnDisk: Long('12288'), empty: false },
    { name: 'local', sizeOnDisk: Long('8192'), empty: false },
    { name: 'soc_lite', sizeOnDisk: Long('8192'), empty: false }
  ],
  totalSize: Long('36864'),
  totalSizeMb: Long('0'),
  ok: 1
}
```

FIGURE 2 – Capture 2 – Insertion anonyme et commande d'administration sur l'instance insecure.

Commentaire d'analyse. Un document `test_insecure` est inséré sans identifiants dans `security_logs`, puis retrouvé avec succès. La commande `db.adminCommand({ listDatabases: 1 })` réussit également. Cela établit la compromission totale possible en mode non authentifié.

```
C:\Users\zguio>docker rm -f mongo_insecure
mongo_insecure

C:\Users\zguio>docker run -d --name mongo_secure -p 127.0.0.1:27017:27017
-e MONGO_INITDB_ROOT_USERNAME=adminSoc -e MONGO_INITDB_ROOT_PASSWORD=Passw0rd_S3cure! mongo:7
902ff3cb76c80d82b9f0f241c808d4908721d7e078b87733b91986c25b932a95

C:\Users\zguio>docker exec -it mongo_secure mongosh
Current Mongosh Log ID: 69da9016734b64a09c8563b0
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS
=2000&appName=mongosh+2.7.0
Using MongoDB:      7.0.30
Using Mongosh:      2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically
(https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

test> use soc_lite
switched to db soc_lite
soc_lite> db.security_logs.find()
MongoServerError[Unauthorized]: Command find requires authentication
```

FIGURE 3 – Capture 3 – Passage à mongo_secure avec publication locale et refus d'accès anonyme.

Commentaire d'analyse. Le conteneur insecure est supprimé puis remplacé par mongo_secure, lié à 127.0.0.1. Une tentative de lecture de security_logs échoue avec Command find requires authentication.

```
C:\Users\zguio>docker exec -it mongo_secure mongosh -u adminSoc -p Passw0rd_S3cure!
Current Mongosh Log ID: 69da90ca1e67db948b8563b0
Connecting to:      mongodb://<credentials>@127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.7.0
Using MongoDB:      7.0.30
Using Mongosh:      2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
  The server generated these startup warnings when booting
  2026-04-11T18:16:12.105+00:00: Using the XFS filesystem is strongly recommended with the Wired
  Tiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
  -----

test> db.adminCommand({listDatabases : 1})
{
  databases: [
    { name: 'admin', sizeOnDisk: Long('102400'), empty: false },
    { name: 'config', sizeOnDisk: Long('12288'), empty: false },
    { name: 'local', sizeOnDisk: Long('73728'), empty: false }
  ],
  totalSize: Long('188416'),
  totalSizeMb: Long('0'),
  ok: 1
}
```

FIGURE 4 – Capture 4 – Validation du compte administrateur sur l'instance sécurisée.

Commentaire d'analyse. Après authentification en tant qu'administrateur, `listDatabases` fonctionne. Cela confirme que la sécurité distingue correctement un client non authentifié d'un client autorisé.

```

C:\Users\zguio>docker run -d --name mongo_manual -p 127.0.0.1:27017:27017 mongo:7 mongod --auth
--bind_ip_all
307819fbc1ff4f56396070022bb43e63da303b3a65b47556b8256692036d65c3

C:\Users\zguio>docker exec -it mongo_manual mongosh
Current Mongosh Log ID: 69da9c68ceefbeba5a8563b0
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS
=2000&appName=mongosh+2.7.0
Using MongoDB:      7.0.30
Using Mongosh:      2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically
(https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

test> use admin
switched to db admin
admin> db.createUser({
  user: "adminSoc",
  pwd: "Passw0rd_S3cure!",
  roles: [{ role: "root", db: "admin" }]
})
{ ok: 1 }
admin> quit()

C:\Users\zguio>docker exec -it mongo_manual mongosh -u adminSoc -p Passw0rd_S3cure!
Current Mongosh Log ID: 69da9cec7fc6e372218563b0
Connecting to:      mongodb://<credentials>@127.0.0.1:27017/?directConnection=true&serverSele
ctionTimeoutMS=2000&appName=mongosh+2.7.0
Using MongoDB:      7.0.30
Using Mongosh:      2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
  The server generated these startup warnings when booting
    2026-04-11T19:09:09.348+00:00: Using the XFS filesystem is strongly recommended with the Wired
Tiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
-----

test> db.adminCommand({listDatabases:1})
{
  databases: [
    { name: 'admin', sizeOnDisk: Long('102400'), empty: false },
    { name: 'config', sizeOnDisk: Long('12288'), empty: false },
    { name: 'local', sizeOnDisk: Long('40960'), empty: false }
  ],
  totalSize: Long('155648'),
  totalSizeMb: Long('0'),
  ok: 1
}

```

FIGURE 5 – Capture 5 – Création manuelle du compte root sur mongo_manual.

Commentaire d'analyse. Cette capture documente un scénario plus avancé : démarrage manuel avec `mongod --auth --bind_ip_all`, création du compte root dans la base `admin`, puis reconnexion réussie.

```
C:\Users\zguio>docker exec -it mongo_manual mongosh -u soc_app -p AppP@ss2024! --authenticationDatabase soc_lite
Current Mongosh Log ID: 69daaf4a06182c432a8563b0
Connecting to:      mongodb://<credentials>@127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&authSource=soc_lite&appName=mongosh+2.7.0
Using MongoDB:      7.0.30
Using Mongosh:      2.7.0

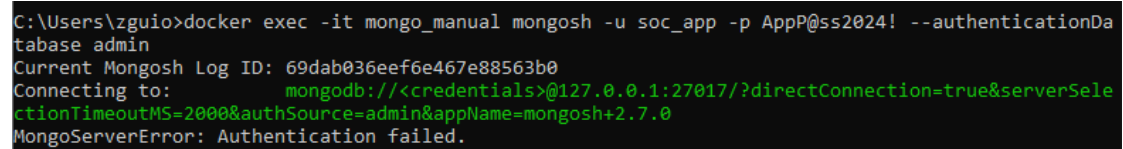
For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

test> use soc_lite
switched to db soc_lite
soc_lite> db.security_logs.find()

soc_lite>
```

FIGURE 6 – Capture 6 – Authentification réussie de `soc_app` avec `authenticationDatabase = soc_lite`.

Commentaire d'analyse. Le compte applicatif se connecte avec la bonne base d'authentification et peut interroger `security_logs`. La capture sert de validation positive du mécanisme d'authentification par base.



```
C:\Users\zguio>docker exec -it mongo_manual mongosh -u soc_app -p AppP@ss2024! --authenticationDatabase admin
Current Mongosh Log ID: 69dab036eef6e467e88563b0
Connecting to:      mongodb://<credentials>@127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&authSource=admin&appName=mongosh+2.7.0
MongoServerError: Authentication failed.
```

FIGURE 7 – Capture 7 – Échec d’authentification de `soc_app` avec la mauvaise authDB.

Commentaire d’analyse. Ici, la même identité applicative est utilisée contre `admin`. MongoDB retourne `Authentication failed`. Cette capture est importante car elle prouve que le paramètre d’authDB n’est pas décoratif : il conditionne réellement l’accès.

```
C:\Users\zguio>docker exec -it mongo_manual mongosh -u adminSoc -p Passw0rd_S3cure!
Current Mongosh Log ID: 69dbcfb94ac1c0996d8563b0
Connecting to:      mongodb://<credentials>@127.0.0.1:27017/?directConnection=true&serverSelecti
onTimeoutMS=2000&appName=mongosh+2.7.0
Using MongoDB:      7.0.30
Using Mongosh:      2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
 2026-04-11T19:09:09.348+00:00: Using the XFS filesystem is strongly recommended with the WiredTig
er storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
-----

test> use soc_lite
switched to db soc_lite
soc_lite> db.createRole({
|   role: "incidentManager",
|   privileges: [
|     {
|       resource: { db: "soc_lite", collection: "incidents" },
|       actions: ["find", "insert", "update", "remove"]
|     }
|   ],
|   roles: []
| })
{ ok: 1 }
```

FIGURE 8 – Capture 8 – Création du rôle personnalisé incidentManager.

Commentaire d’analyse. La commande `db.createRole()` définit un rôle limité à la collection `incidents` avec les actions `find`, `insert`, `update` et `remove`. C’est la base du profil manager.

```
soc_lite> db.security_logs.insertMany([
  {
    timestamp: ISODate("2026-03-23T08:15:00Z"),
    source_ip: "10.0.1.45",
    event_type: "login_failed",
    severity: "warning",
    message: "Échec d'authentification SSH - utilisateur root",
    hostname: "srv-web-01"
  },
  {
    timestamp: ISODate("2026-03-23T08:15:03Z"),
    source_ip: "10.0.1.45",
    event_type: "login_failed",
    severity: "warning",
    message: "Échec d'authentification SSH - utilisateur admin",
    hostname: "srv-web-01"
  },
  {
    timestamp: ISODate("2026-03-23T08:16:30Z"),
    source_ip: "10.0.1.45",
    event_type: "login_success",
    severity: "critical",
    message: "Connexion SSH réussie après multiples échecs",
    hostname: "srv-web-01"
  }
])
db.incidents.insertMany([
  {
    incident_id: "INC-2026-0042",
    title: "Brute-force SSH détecté sur srv-web-01",
    severity: "high",
    status: "open",
    created_at: ISODate("2026-03-23T08:20:00Z"),
    assigned_to: "analyst_dupont",
    description: "Multiples tentatives de connexion SSH suivies d'un succès."
  },
  {
    incident_id: "INC-2026-0041",
    title: "Scan de ports depuis 192.168.5.200",
    severity: "medium",
    status: "investigating",
    created_at: ISODate("2026-03-22T14:05:00Z"),
    assigned_to: "analyst_martin",
    description: "Scan SYN détecté sur la plage 1-1024."
  }
])
db.users.insertMany([
  {
    username: "analyst_dupont",
    email: "dupont@entreprise.fr",
    department: "SOC",
    clearance_level: "confidential"
  },
  {
    username: "analyst_martin",
    email: "martin@entreprise.fr",
    department: "SOC",
    clearance_level: "confidential"
  }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69dbd1c64ac1c0996d8563b6'),
    '1': ObjectId('69dbd1c64ac1c0996d8563b7')
  }
}
```

FIGURE 9 – Capture 9 – Insertion du jeu de données de test dans `security_logs`, `incidents` et `users`.

Commentaire d'analyse. La capture montre l'insertion de plusieurs journaux SSH, de deux incidents et de deux utilisateurs. Elle fournit le socle de données nécessaire aux vérifications RBAC ultérieures.

```
soc_lite> db.createUser({
|   user: "soc_analyst_ro",
|   pwd: "AnalystR0@2026!",
|   roles: [
|     { role: "read", db: "soc_lite" }
|   ]
| })
{ ok: 1 }
soc_lite> db.createUser({
|   user: "soc_manager",
|   pwd: "Mgr$ecure2026!",
|   roles: [
|     { role: "incidentManager", db: "soc_lite" }
|   ]
| })
{ ok: 1 }
```

FIGURE 10 – Capture 10 – Création de soc_analyst_ro et soc_manager.

Commentaire d'analyse. Le compte analyste reçoit le rôle built-in `read` sur `soc_lite`, tandis que le compte manager reçoit le rôle personnalisé `incidentManager`.

```
soc_lite> db.getUsers()
{
  users: [
    {
      _id: 'soc_lite.soc_analyst_ro',
      userId: UUID('f848cdd5-de4a-4056-bca8-b319389d8a38'),
      user: 'soc_analyst_ro',
      db: 'soc_lite',
      roles: [ { role: 'read', db: 'soc_lite' } ],
      mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
    },
    {
      _id: 'soc_lite.soc_app',
      userId: UUID('3300ecf5-c4f2-4dd4-8b7b-d6decd82d459'),
      user: 'soc_app',
      db: 'soc_lite',
      roles: [ { role: 'readWrite', db: 'soc_lite' } ],
      mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
    },
    {
      _id: 'soc_lite.soc_manager',
      userId: UUID('87479a83-bb75-41ed-a9bc-63bfcfa85db2'),
      user: 'soc_manager',
      db: 'soc_lite',
      roles: [ { role: 'incidentManager', db: 'soc_lite' } ],
      mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
    }
  ],
  ok: 1
}
soc_lite> db.getRoles()
{
  roles: [
    {
      _id: 'soc_lite.incidentManager',
      role: 'incidentManager',
      db: 'soc_lite',
      roles: [],
      isBuiltin: false,
      inheritedRoles: []
    }
  ],
  ok: 1
}
```

FIGURE 11 – Capture 12 – Vérification des utilisateurs et rôles présents dans soc_lite.

Commentaire d’analyse. La sortie de `db.getUsers()` confirme la présence de `soc_analyst_ro`, `soc_app` et `soc_manager`. La sortie de `db.getRoles()` confirme la présence du rôle personnalisé `incidentManager`.

```
soc_lite> db.getRoles({showBuiltinRoles:true})
{
  roles: [
    {
      role: 'readWrite',
      db: 'soc_lite',
      isBuiltin: true,
      roles: [],
      inheritedRoles: []
    },
    {
      role: 'userAdmin',
      db: 'soc_lite',
      isBuiltin: true,
      roles: [],
      inheritedRoles: []
    },
    {
      role: 'enableSharding',
      db: 'soc_lite',
      isBuiltin: true,
      roles: [],
      inheritedRoles: []
    },
    {
      role: 'read',
      db: 'soc_lite',
      isBuiltin: true,
      roles: [],
      inheritedRoles: []
    },
    {
      role: 'dbOwner',
      db: 'soc_lite',
      isBuiltin: true,
      roles: [],
      inheritedRoles: []
    },
    {
      role: 'dbAdmin',
      db: 'soc_lite',
      isBuiltin: true,
      roles: [],
      inheritedRoles: []
    },
    {
      _id: 'soc_lite.incidentManager',
      role: 'incidentManager',
      db: 'soc_lite',
      roles: [],
      isBuiltin: false,
      inheritedRoles: []
    }
  ],
  ok: 1
}
```

FIGURE 12 – Capture 13 – Visualisation des rôles built-in et du rôle personnalisé.

Commentaire d’analyse. La commande `db.getRoles({showBuiltinRoles:true})` rappelle que `read` et `readWrite` sont des rôles globaux à l’échelle de la base. Cette capture aide à expliquer les débordements de privilèges observés ensuite.

```

soc_lite> quit()

C:\Users\zguio>docker exec -it mongo_manual mongosh -u soc_analyst_ro -p AnalystR0@2026! --authenticationDatabase soc_lite
Current Mongosh Log ID: 69dbd4f6c3685fd6a8563b0
Connecting to:  mongodb://<credentials>@127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&authSource=soc_1
ite&appName=mongosh+2.7.0
Using MongoDB: 7.0.30
Using Mongosh: 2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

test> use soc_lite
switched to db soc_lite
soc_lite> db.security_logs.find().limit(2)
[
  {
    _id: ObjectId('69dbd1c64ac1c0996d8563b1'),
    timestamp: ISODate('2026-03-23T08:15:00.000Z'),
    source_ip: '10.0.1.45',
    event_type: 'login_failed',
    severity: 'warning',
    message: "Échec d'authentification SSH - utilisateur root",
    hostname: 'srv-web-01'
  },
  {
    _id: ObjectId('69dbd1c64ac1c0996d8563b2'),
    timestamp: ISODate('2026-03-23T08:15:03.000Z'),
    source_ip: '10.0.1.45',
    event_type: 'login_failed',
    severity: 'warning',
    message: "Échec d'authentification SSH - utilisateur admin",
    hostname: 'srv-web-01'
  }
]
soc_lite> db.incidents.find({ severity: "high" })
[
  {
    _id: ObjectId('69dbd1c64ac1c0996d8563b4'),
    incident_id: 'INC-2026-0042',
    title: 'Brute-force SSH détecté sur srv-web-01',
    severity: 'high',
    status: 'open',
    created_at: ISODate('2026-03-23T08:20:00.000Z'),
    assigned_to: 'analyst_dupont',
    description: "Multiples tentatives de connexion SSH suivies d'un succès."
  }
]
soc_lite> db.security_logs.insertOne({ event_type: "test_write", message: "tentative" })
MongoServerError[Unauthorized]: not authorized on soc_lite to execute command { insert: "security_logs", documents: [ { event_type: "
test_write", message: "tentative", _id: ObjectId('69dbd52fc3685fd6a8563b1') } ], ordered: true, lsid: { id: UUID("7f045a72-4652-4273
-9ff0-048abe8b1662") }, $db: "soc_lite" }
soc_lite> db.incidents.deleteOne({ incident_id: "INC-2026-0042" })
MongoServerError[Unauthorized]: not authorized on soc_lite to execute command { delete: "incidents", deletes: [ { q: { incident_id: "
INC-2026-0042" }, limit: 1 } ], ordered: true, lsid: { id: UUID("7f045a72-4652-4273-9ff0-048abe8b1662") }, $db: "soc_lite" }
soc_lite> db.users.find()
[
  {
    _id: ObjectId('69dbd1c64ac1c0996d8563b6'),
    username: 'analyst_dupont',
    email: 'dupont@entreprise.fr',
    department: 'SOC',
    clearance_level: 'confidential'
  },
  {
    _id: ObjectId('69dbd1c64ac1c0996d8563b7'),
    username: 'analyst_martin',
    email: 'martin@entreprise.fr',
    department: 'SOC',
    clearance_level: 'confidential'
  }
]

```

FIGURE 13 – Capture 14 – Tests du compte soc_analyst_ro.

Commentaire d’analyse. Les lectures de logs et d’incidents sont autorisées, l’insertion et la suppression sont refusées, mais `db.users.find()` retourne les documents. La capture est centrale car elle matérialise l’écart entre la politique souhaitée et le comportement réel du rôle built-in read.

```

C:\Users\zguio>docker exec -it mongo_manual mongosh -u soc_manager -p Mgr$ecure2026! --authenticationDatabase soc_lite
Current Mongosh Log ID: 69dbdcb4a59d53e5d8563b0
Connecting to:      mongodb://<credentials>@127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&authSource=soc_1
ite&appName=mongosh+2.7.0
Using MongoDB:      7.0.30
Using Mongosh:       2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

test> use soc_lite
switched to db soc_lite
soc_lite> db.incidents.find()
[
  {
    _id: ObjectId('69dbd1c64ac1c0996d8563b4'),
    incident_id: 'INC-2026-0042',
    title: 'Brute-force SSH détecté sur srv-web-01',
    severity: 'high',
    status: 'open',
    created_at: ISODate('2026-03-23T08:20:00.000Z'),
    assigned_to: 'analyst_dupont',
    description: "Multiples tentatives de connexion SSH suivies d'un succès."
  },
  {
    _id: ObjectId('69dbd1c64ac1c0996d8563b5'),
    incident_id: 'INC-2026-0041',
    title: 'Scan de ports depuis 192.168.5.200',
    severity: 'medium',
    status: 'investigating',
    created_at: ISODate('2026-03-22T14:05:00.000Z'),
    assigned_to: 'analyst_martin',
    description: 'Scan SYN détecté sur la plage 1-1024.'
  }
]
soc_lite> db.incidents.updateOne(
  { incident_id: "INC-2026-0042" },
  { $set: { status: "investigating", assigned_to: "manager_leroy" } }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
soc_lite> db.security_logs.find().limit(1)
MongoServerError[Unauthorized]: not authorized on soc_lite to execute command { find: "security_logs", filter: {}, limit: 1, lsid: {
id: UUID("3f121165-b359-49a1-b60f-d88743e1ac43") }, $db: "soc_lite" }
soc_lite> db.users.find()
MongoServerError[Unauthorized]: not authorized on soc_lite to execute command { find: "users", filter: {}, lsid: { id: UUID("3f121165
-b359-49a1-b60f-d88743e1ac43") }, $db: "soc_lite" }
soc_lite> db.adminCommand({listDatabases: 1})
{
  databases: [ { name: 'soc_lite', sizeOnDisk: Long('139264'), empty: false } ],
  totalSize: Long('139264'),
  totalSizeMb: Long('0'),
  ok: 1
}

```

FIGURE 14 – Capture 15 – Tests du compte `soc_manager`.

Commentaire d'analyse. Le compte manager lit les incidents, modifie le statut d'un incident avec succès, mais ne peut ni lire `security_logs`, ni lire `users`, ni administrer l'instance. C'est la mise en œuvre la plus propre du moindre privilège dans l'ensemble des preuves.

```
C:\Users\zguio>docker exec -it mongo_manual mongosh "mongodb://soc_app:AppP@ss2026!@localhost:27017/soc_lite"
Current Mongosh Log ID: 69dbe0e7b094ea0c6f8563b0
Connecting to:      mongodb://<credentials>@localhost:27017/soc_lite?directConnection=true&serverSelectionTimeoutMS=2000&appName=
Mongosh+2.7.0
Using MongoDB:      7.0.30
Using Mongosh:      2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

soc_lite> db.security_logs.insertOne({
  |   timestamp: new Date(),
  |   source_ip: "10.0.2.100",
  |   event_type: "firewall_drop",
  |   severity: "info",
  |   message: "Paquet TCP bloqué par règle DROP"
  | })
  {
    acknowledged: true,
    insertedId: ObjectId('69dbe10db094ea0c6f8563b1')
  }
soc_lite> db.incidents.find({ status: "open" })
soc_lite> db.users.find()
[
  {
    _id: ObjectId('69dbd1c64ac1c0996d8563b6'),
    username: 'analyst_dupont',
    email: 'dupont@entreprise.fr',
    department: 'SOC',
    clearance_level: 'confidential'
  },
  {
    _id: ObjectId('69dbd1c64ac1c0996d8563b7'),
    username: 'analyst_martin',
    email: 'martin@entreprise.fr',
    department: 'SOC',
    clearance_level: 'confidential'
  }
]
soc_lite> db.adminCommand({ listDatabases: 1 })
{
  databases: [ { name: 'soc_lite', sizeOnDisk: Long('172032'), empty: false } ],
  totalSize: Long('172032'),
  totalSizeMb: Long('0'),
  ok: 1
}
```

FIGURE 15 – Capture 16 – Tests du compte `soc_app`.

Commentaire d'analyse. Le compte applicatif insère un événement `firewall_drop` et lit les incidents ouverts. En revanche, il peut aussi lire `users`, ce qui révèle la limite du rôle built-in `readWrite`. L'administration de l'instance reste heureusement refusée.